Capitulo 4. Historial de cambios y ramas en Git

1. **Introducción**

   Git permite consultar el historial de un proyecto para conocer los cambios realizados, quien los efectuó y en qué momento se guardaron. También permite crear ramas, que son líneas de trabajo independientes dentro de un mismo repositorio.
   Estas herramientas son útiles para organizar el desarrollo de un proyecto, revisar modificaciones anteriores y trabajar en nuevas funciones sin afectar la versión principal del repositorio.

2. **Consultar el historial con git log**

   El comando git log muestra el historial de confirmaciones o commits realizados en el repositorio.
   Git log
   La información que muestra git generalmente incluye:
   - El identificador único del commit, conocido como hash
   - El nombre y correo del autor.
   - La fecha y hora de confirmación.
   - El mensaje descriptivo del commit.

   Un ejemplo de salida es el siguiente:

   Commit a1b2c3d4e5f6g7h8i9j0

   Author: Danna Vanessa Devia Mendoza correo@ejemplo.com

   Date: Mon Oct 13 10:30:00 2026 –0500

   Agrega el Capítulo 1 sobre introducción a Git

3. **Versiones resumidas del historial**

   Cuando el repositorio tiene muchos commits, la salida git log puede ser extensa. Para ver una versión más corta, se puede utilizar:
   Git log –oneline
   Este comando muestra cada confirmación en una sola línea, incluyendo el identificador abreviado y el mensaje del commit.
   También se puede limitar la cantidad de commits mostrados. Por ejemplo, para ver los últimos cinco:
   Git log –oneline –5

4. **Comparar cambios con git diff**

   El comando git diff permite ver las diferencias entre la versión actual de los archivos y la última versión confirmada.

Git diff

Si se desea comparar los cambios que ya fueron preparados con git add, se utiliza:

Git diff –staged

Este comando es útil antes de realizar un commit, porque permite revisar exactamente que líneas fueron agregadas, modificadas o eliminadas.

5. **¿Qué es una rama?**

   Una rama o branch es una línea independiente de desarrollo dentro de un repositorio. La rama principal suele llamarse main o master, según la configuración del repositorio.

   Las ramas permiten realizar cambios, pruebas o nuevas secciones de un proyecto sin modificar directamente la rama principal. Una vez que los cambios están listos, pueden integrarse nuevamente a la rama principal.

6. **Ver las ramas del repositorio**

   Para consultar las ramas existentes en el repositorio local, se utiliza:

   Git branch

   La rama activa aparece marcada con un asterisco

   * main

   Para ver también las ramas del repositorio remoto, se pueden usar:

   Git branch –a

7. **Crear una rama nueva**

   Para crear una rama nueva, se utiliza el comando:

   Git branch nombre-de-la-rama

   Por ejemplo:

   Git branch capitulo-4

   Este comando crea la rama, pero no cambia automáticamente a ella.

8. **Cambiar de rama**

   Para cambiar de rama existente, se puede utilizar:

   Git checkout nombre-de-la-rama

   Por ejemplo:

   Git checkout capitulo-4

   También existe el comando más actual git switch:

   Git switch capitulo-4

   Ambos comandos permiten moverse entre ramas; sin embargo, git switch fue credo específicamente para cambiar de rama y ofrece una sintaxis más clara.

9. **Crear y cambiar de rama al mismo tiempo**

   Es posible crear una rama y ubicarse directamente en ella con un solo comando:

   Git checkout –b nombre-de-la-rama

   Por ejemplo:

Git checkout –b capitulo-4

Después de ejecutar el comando, Git crea la rama y cambia automáticamente a ella.

10. Buenas prácticas con ramas

Para mantener un repositorio organizado, se recomiendo:

- Utilizar nombres claros relacionados con el cambio o la sección desarrollo.
- Crear una rama para cada tema, capitulo o corrección importante.
- Evitar realizar cambios grandes directamente en la rama principal.
- Mantener mensajes de commit descriptivos en cada rama.

11. Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. Disponible en: https://git-scm.com/book/es/v2